

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
KHOA CÔNG NGHỆ VÀ KỸ THUẬT



**XÂY DỰNG WEBSITE BÁN DỊCH VỤ CLOUD
VỚI ASP.NET CORE VÀ NEXT.JS**

Ngành: Khoa học máy tính

Tên học phần: Phát triển phần mềm hướng đối tượng

Mã học phần: IN421101

Nhóm thực hiện: Nhóm 7
Lớp: 002341480101A
Giảng viên hướng dẫn: Lê Minh Hiếu

Đồng Tháp, tháng 8 năm 2026

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến **thầy Lê Minh Hiếu** đã tận tình giảng dạy, truyền đạt những kiến thức quý báu về Phát triển phần mềm hướng đối tượng trong suốt học kỳ vừa qua. Những bài giảng về Clean Architecture, Design Patterns, Unit Testing và quy trình CI/CD đã trở thành nền tảng vững chắc để nhóm có thể hoàn thành đồ án này. Hiếu

Chúng em cũng xin cảm ơn **Khoa Công nghệ và Kỹ thuật – Trường Đại học Đồng Tháp** đã tạo điều kiện thuận lợi về cơ sở vật chất và môi trường học tập, giúp chúng em có không gian phát triển kỹ năng lập trình thực tế.

Cuối cùng, chúng em xin gửi lời cảm ơn đến tất cả các thành viên trong nhóm đã cùng nhau nỗ lực, phối hợp chặt chẽ và không ngừng học hỏi để hoàn thành dự án đúng tiến độ.

Mặc dù đã cố gắng hết sức, báo cáo chắc chắn không tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự góp ý từ quý thầy cô để hoàn thiện hơn trong tương lai.

Đồng Tháp, tháng 8 năm 2026

Nhóm 7

DANH SÁCH THÀNH VIÊN NHÓM

STT	Họ và tên	Mã sinh viên	Ghi chú
1	Nguyễn Trung Hậu	0023411914	Nhóm trưởng
2	Lương Quang Khải	0022412375	
3	Võ Phụng Tiên	0022410955	
4	Phan Thành Vinh	0023412001	

MỤC LỤC

LỜI CẢM ƠN	1
DANH SÁCH THÀNH VIÊN NHÓM	2
MỤC LỤC TỪ VIẾT TẮT	6
MỞ ĐẦU	i
I CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG	1
1.1 Tổng quan về Điện toán đám mây (Cloud Computing)	1
1.2 Nền tảng và Công nghệ phía Backend	1
1.2.1 ASP.NET Core và C#	1
1.2.2 Entity Framework Core (EF Core)	1
1.2.3 SQL Server 2022	2
1.3 Nền tảng và Công nghệ phía Frontend	2
1.3.1 Next.js và React	2
1.3.2 Tailwind CSS	2
1.3.3 Axios và Quản lý Token	2
1.4 Xác thực và Bảo mật với JWT (JSON Web Token)	2
1.5 Các nguyên lý thiết kế SOLID	3
1.6 Các công cụ hỗ trợ phát triển	3
II PHÂN TÍCH YÊU CẦU HỆ THỐNG	4
2.1 Xác định tác nhân (Actors)	4
2.2 Yêu cầu chức năng	4
2.2.1 Chức năng trang công khai (Landing Page)	4
2.2.2 Chức năng trang quản trị (Admin Dashboard)	5
2.3 Yêu cầu phi chức năng	5
2.4 Sơ đồ Use Case (Ca sử dụng)	5
2.5 Sơ đồ Hoạt động (Activity Diagram)	7

III THIẾT KẾ HỆ THỐNG	9
3.1 Thiết kế Kiến trúc (Architecture Design)	9
3.2 Thiết kế Cơ sở dữ liệu (Database Design)	9
3.2.1 Sơ đồ thực thể liên kết (ERD)	9
3.2.2 Mô tả các thực thể (Entities) chính	9
3.3 Thiết kế RESTful API	9
3.4 Thiết kế Mẫu thiết kế (Design Patterns)	12
3.4.1 Repository Pattern	12
3.4.2 Unit of Work Pattern	12
3.4.3 Middleware Pattern	12
3.4.4 Dependency Injection (DI)	13
3.5 Sơ đồ Tuần tự (Sequence Diagram)	13
IV XÂY DỰNG VÀ KIỂM THỬ HỆ THỐNG	15
4.1 Xây dựng Giao diện người dùng (UI)	15
4.1.1 Cấu trúc thư mục Frontend	15
4.1.2 Giao diện Landing Page công khai	15
4.1.3 Giao diện Quản trị (Admin Dashboard)	17
4.2 Kiểm thử đơn vị (Unit Testing)	17
4.2.1 Tổng quan kết quả	17
4.2.2 Phương pháp Mock	17
4.3 Triển khai liên tục (CI/CD) và Docker	19
4.3.1 Luồng làm việc với GitHub Actions (CI)	19
4.3.2 Đóng gói ứng dụng với Docker	19
4.3.3 Quy trình làm việc nhóm với Git	19
V KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	20
5.1 Đánh giá kết quả đạt được	20
5.1.1 Khía cạnh Kỹ thuật	20
5.1.2 Khía cạnh Nghiệp vụ	20
5.1.3 Khía cạnh Quy trình	20
5.2 Bảng tổng hợp kết quả đối chiếu đề bài	21
5.3 Phân công công việc	21
5.4 Hạn chế và Hướng phát triển trong tương lai	21
PHỤ LỤC	24

MỤC LỤC HÌNH ẢNH

II.1	Sơ đồ Use Case tổng quát của hệ thống Website bán dịch vụ Cloud	6
II.2	Sơ đồ Hoạt động nghiệp vụ Đặt hàng dịch vụ	8
III.1	Sơ đồ khối Clean Architecture trong dự án	10
III.2	Sơ đồ thực thể liên kết (ERD)	11
III.3	Sơ đồ Tuần tự luồng xử lý xác thực đăng nhập cấp phát JWT	14
IV.1	Giao diện trang chủ (Landing Page)	16
IV.2	Giao diện trang Bảng giá – Chuyển đổi Tháng/Năm và hiển thị mã QR	16
IV.3	Giao diện Bảng điều khiển Quản trị viên (Admin Dashboard)	17
IV.4	Kết quả chạy 16 Unit Test toàn bộ PASSED	18

MỤC LỤC TỪ VIẾT TẮT

- **API** (Application Programming Interface): Giao diện lập trình ứng dụng.
- **CI/CD** (Continuous Integration/Continuous Deployment): Tích hợp liên tục và Triển khai liên tục.
- **ERD** (Entity-Relationship Diagram): Sơ đồ thực thể liên kết.
- **JWT** (JSON Web Token): Tiêu chuẩn mở an toàn cho việc truyền thông tin.
- **ORM** (Object-Relational Mapping): Kỹ thuật ánh xạ đối tượng với dữ liệu quan hệ.
- **SOLID**: Tập hợp 5 nguyên lý thiết kế hướng đối tượng.
- **VPS** (Virtual Private Server): Máy chủ riêng ảo.
- **SLA** (Service Level Agreement): Cam kết mức độ dịch vụ.

MỞ ĐẦU

1. Bối cảnh đề tài

Sự phát triển mạnh mẽ của công nghệ điện toán đám mây (Cloud Computing) đã thúc đẩy quá trình chuyển đổi số của các doanh nghiệp. Các dịch vụ như VPS (Máy chủ ảo), Hosting, Domain, Email doanh nghiệp hay bảo mật Firewall không còn xa lạ mà trở thành nền tảng thiết yếu cho bất kỳ doanh nghiệp nào muốn hiện diện và vận hành trên không gian mạng. Để đáp ứng nhu cầu đó, các nhà cung cấp dịch vụ Cloud cần có một hệ thống website chuyên nghiệp, ổn định để quảng bá dịch vụ, hỗ trợ khách hàng đăng ký và quản trị hệ thống một cách hiệu quả.

Nhận thấy tầm quan trọng của hệ thống website này, nhóm chúng em đã quyết định chọn đề tài: "**Xây dựng Website bán dịch vụ Cloud**". Đồ án tập trung vào việc áp dụng các kiến thức cốt lõi của học phần Phát triển phần mềm hướng đối tượng để thiết kế và xây dựng một hệ thống hoàn chỉnh từ Backend đến Frontend, đáp ứng các tiêu chuẩn công nghiệp hiện đại.

2. Mục tiêu nghiên cứu

Đồ án đặt ra các mục tiêu chính sau đây:

- Ứng dụng mô hình **Clean Architecture** cùng các nguyên lý **SOLID** và **Design Patterns** (như Repository, Unit of Work) vào xây dựng Backend Web API bằng nền tảng ASP.NET Core 9.
- Phát triển giao diện người dùng hiện đại, tốc độ cao (Frontend) bằng công nghệ **Next.js** có khả năng hiển thị tương thích trên nhiều thiết bị (responsive).
- Triển khai các tính năng cốt lõi cho một website bán dịch vụ: Hiển thị bảng giá dịch vụ (VPS, Hosting, v.v.), tiếp nhận đơn đặt hàng, quản trị người dùng, hệ thống bảo mật bằng JWT và quản lý mã khuyến mãi.
- Vận dụng các quy trình làm việc nhóm hiệu quả thông qua Git/GitHub, thực hành viết **Unit Test**, xây dựng pipeline **CI/CD** và đóng gói ứng dụng bằng **Docker**.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu:

- Kiến trúc phần mềm và các mẫu thiết kế hướng đối tượng.
- Nền tảng .NET Core và công nghệ Next.js (React).
- Các quy trình phát triển và kiểm thử phần mềm tự động.

Phạm vi nghiên cứu: Đồ án tập trung xây dựng trang giới thiệu công khai (Landing Page) cho khách hàng và khu vực quản trị (Admin Dashboard) dành riêng cho quản trị viên với các nghiệp vụ quản lý gói dịch vụ, đơn hàng và tin tức nội bộ.

4. Cấu trúc của báo cáo

Báo cáo được chia thành 5 chương chính:

- **Chương I: Cơ sở lý thuyết và công nghệ áp dụng** – Trình bày tổng quan về điện toán đám mây, các công nghệ Backend (ASP.NET Core, EF Core), Frontend (Next.js, Tailwind CSS) và cơ chế xác thực JWT.
- **Chương II: Phân tích yêu cầu hệ thống** – Xác định tác nhân, yêu cầu chức năng (Landing Page và Admin Dashboard), yêu cầu phi chức năng, sơ đồ Use Case và Activity Diagram.
- **Chương III: Thiết kế hệ thống** – Mô tả kiến trúc Clean Architecture 4 tầng, thiết kế cơ sở dữ liệu (ERD), các Design Patterns (Repository, Unit of Work) và Sequence Diagram.
- **Chương IV: Xây dựng và kiểm thử hệ thống** – Trình bày giao diện người dùng thực tế, kết quả Unit Test (16 test cases) và hệ thống CI/CD, Docker.
- **Chương V: Kết luận và hướng phát triển** – Đánh giá kết quả đạt được, phân công công việc của từng thành viên và định hướng phát triển tương lai.

CHƯƠNG I: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ỨNG DỤNG

1.1 Tổng quan về Điện toán đám mây (Cloud Computing)

Điện toán đám mây là mô hình cung cấp các tài nguyên máy tính (máy chủ, mạng, lưu trữ, cơ sở dữ liệu, phần mềm) qua Internet (“đám mây”) nhằm cung cấp sự đổi mới nhanh hơn, tài nguyên linh hoạt và lợi thế kinh tế theo quy mô.

Trong khuôn khổ của hệ thống bán dịch vụ Cloud, các khái niệm dịch vụ chính được hệ thống cung cấp bao gồm:

- **VPS (Virtual Private Server):** Máy chủ riêng ảo được phân chia từ một máy chủ vật lý, cung cấp tài nguyên độc lập, có hệ điều hành riêng và cho phép toàn quyền quản trị (Root/Administrator).
- **Web Hosting:** Dịch vụ lưu trữ dữ liệu website trên máy chủ đã được cấu hình sẵn môi trường (web server, database server), giúp khách hàng dễ dàng đưa website lên Internet mà không cần kiến thức quản trị máy chủ sâu sắc.
- **Tên miền (Domain Name):** Địa chỉ định danh của một website trên mạng Internet, đóng vai trò như “địa chỉ nhà” thay thế cho dãy địa chỉ IP phức tạp.

1.2 Nền tảng và Công nghệ phía Backend

1.2.1 ASP.NET Core và C#

ASP.NET Core [3] là một framework mã nguồn mở, đa nền tảng do Microsoft phát triển dùng để xây dựng các ứng dụng web hiện đại, các dịch vụ IoT và các backend dùng cho ứng dụng di động. ASP.NET Core có hiệu năng rất cao, thiết kế theo mô hình mô-đun (modular) và hỗ trợ tích hợp sẵn Dependency Injection.

Trong dự án này, ASP.NET Core 9 Web API được sử dụng để phát triển RESTful API phục vụ cho quá trình trao đổi dữ liệu với Frontend. API được chia thành hai nhóm chính: `Public API` (không yêu cầu đăng nhập) và `Admin API` (yêu cầu xác thực JWT và phân quyền Role).

1.2.2 Entity Framework Core (EF Core)

EF Core là một Object-Relational Mapper (ORM) nhẹ, mã nguồn mở, giúp lập trình viên C# làm việc với cơ sở dữ liệu quan hệ bằng cách sử dụng các đối tượng .NET mạnh mẽ mà không cần phải viết phần lớn các câu truy vấn cơ sở dữ liệu thông thường. Hệ thống sử dụng EF Core kết hợp với SQL Server 2022 để lưu trữ và truy xuất dữ liệu. Quá trình quản lý sự thay đổi cấu trúc bảng được thực hiện thông qua cơ chế **Migrations** tự động.

1.2.3 SQL Server 2022

Microsoft SQL Server là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) được sử dụng rộng rãi trong môi trường doanh nghiệp. Trong dự án, SQL Server 2022 phiên bản Express được triển khai thông qua Docker container, giúp nhất quán môi trường phát triển giữa các thành viên trong nhóm.

1.3 Nền tảng và Công nghệ phía Frontend

1.3.1 Next.js và React

Next.js [4] là một framework của React giúp xây dựng các ứng dụng web tối ưu hóa SEO thông qua khả năng Server-Side Rendering (SSR) và Static Site Generation (SSG). Next.js sử dụng kiến trúc App Router mới (từ phiên bản 13 trở lên), giúp cấu trúc thư mục logic hơn và hỗ trợ các tính năng React Server Components.

Trong dự án, Next.js được sử dụng để xây dựng toàn bộ giao diện người dùng bao gồm: Landing Page công khai (10 trang), Admin Dashboard (8 trang quản trị), và Customer Dashboard.

1.3.2 Tailwind CSS

Tailwind CSS là một utility-first CSS framework giúp xây dựng các giao diện người dùng nhanh chóng mà không cần phải thoát ra khỏi tệp HTML/JSEX. Nó cung cấp các class mức độ thấp để kết hợp lại tạo thành bất kỳ thiết kế tùy chỉnh nào. Giao diện trang chủ và trang quản trị của dự án được thiết kế hoàn toàn bằng Tailwind CSS, mang lại vẻ ngoài hiện đại và thân thiện với thiết bị di động (Responsive Design).

1.3.3 Axios và Quản lý Token

Thư viện Axios được sử dụng để thực hiện các HTTP Request từ Frontend đến Backend API. Hệ thống cấu hình một `ApiClient` tập trung với:

- `baseUrl`: trỏ đến địa chỉ Backend API (`http://localhost:5023/api`).
- `Interceptor Request`: tự động gắn JWT Token vào tiêu đề `Authorization` của mọi request.
- `Interceptor Response`: xử lý lỗi 401 (Unauthorized) bằng cách xóa token cũ và chuyển hướng về trang đăng nhập.

1.4 Xác thực và Bảo mật với JWT (JSON Web Token)

JSON Web Token (JWT) là một tiêu chuẩn mở (RFC 7519) định nghĩa một cách nhỏ gọn và tự chứa để truyền tải thông tin an toàn giữa các bên dưới dạng một đối tượng JSON. Thông tin này có thể được xác minh và tin cậy vì nó được ký điện tử.

Trong hệ thống, JWT được ứng dụng làm cơ chế xác thực chính (Stateless Authentication):

1. Quản trị viên gửi yêu cầu đăng nhập (email và mật khẩu) lên Backend qua endpoint `POST /api/auth/login`.
2. Backend kiểm tra mật khẩu đã mã hóa bằng thuật toán **BCrypt**, nếu đúng sẽ sinh ra một chuỗi JWT (AccessToken) có chứa thông tin cơ bản (ID người dùng, Role) cùng với chữ ký bí mật, và một `RefreshToken` để gia hạn phiên làm việc.

3. Frontend lưu token vào Cookie và gửi kèm trong tiêu đề `Authorization: Bearer <token>` của các yêu cầu tiếp theo.
4. Backend xác minh chữ ký của JWT thông qua Middleware; nếu hợp lệ, yêu cầu được cấp phép thực hiện theo phân quyền tương ứng (Admin hoặc Editor).

1.5 Các nguyên lý thiết kế SOLID

SOLID là tập hợp 5 nguyên lý thiết kế hướng đối tượng giúp tạo ra phần mềm linh hoạt, dễ bảo trì và mở rộng:

- **S – Single Responsibility Principle:** Mỗi lớp chỉ chịu trách nhiệm cho một chức năng duy nhất. Ví dụ: `OrderService` chỉ xử lý logic đơn hàng, `JwtService` chỉ xử lý việc tạo/xác thực token.
- **O – Open/Closed Principle:** Hệ thống mở cho việc mở rộng nhưng đóng cho việc sửa đổi. Ví dụ: thêm một dịch vụ mới (SSL, Firewall) chỉ cần tạo thêm bản ghi trong bảng `ServiceCategory` mà không cần sửa code.
- **L – Liskov Substitution Principle:** Các lớp triển khai (Implementation) có thể thay thế cho lớp cha (Interface) mà không ảnh hưởng logic. Ví dụ: `OrderService` triển khai giao diện `IOrderService`, có thể thay bằng `MockOrderService` khi test.
- **I – Interface Segregation Principle:** Hệ thống sử dụng 9 giao diện chuyên biệt (`IOrderService`, `ICategoryService`, `INewsArticleService`...) thay vì một giao diện lớn chứa tất cả.
- **D – Dependency Inversion Principle:** Các lớp cấp cao (Controller, Service) phụ thuộc vào giao diện trừu tượng (Interface) thay vì lớp cụ thể. Việc “nối dây” (wiring) được thực hiện tại `Program.cs` thông qua Dependency Injection container có sẵn của ASP.NET Core.

1.6 Các công cụ hỗ trợ phát triển

- **Git & GitHub:** Quản lý phiên bản mã nguồn, làm việc nhóm qua Pull Request.
- **Docker & Docker Compose:** Đóng gói ứng dụng và cơ sở dữ liệu vào container.
- **GitHub Actions:** Pipeline CI tự động build và chạy unit test khi push/PR.
- **xUnit & Moq:** Framework kiểm thử đơn vị và thư viện mock cho .NET.
- **Swagger (OpenAPI):** Tự động tạo tài liệu API tương tác cho Backend.
- **QRCode:** Thư viện .NET sinh mã QR Code cho từng gói dịch vụ.
- **ClosedXML:** Thư viện .NET xuất dữ liệu ra file Excel (.xlsx).

CHƯƠNG II: PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1 Xác định tác nhân (Actors)

Trong hệ thống website cung cấp dịch vụ Cloud, có hai tác nhân chính tham gia tương tác với hệ thống:

- **Khách hàng (Customer):** Là người dùng truy cập vào website để xem thông tin dịch vụ, so sánh bảng giá, đặt mua dịch vụ (VPS, Hosting, Domain, v.v.) và đăng ký trở thành đối tác liên kết (Affiliate).
- **Quản trị viên (Admin/Editor):** Là nhân viên của công ty cung cấp dịch vụ, có nhiệm vụ cấu hình bảng giá, quản lý các danh mục dịch vụ, xử lý/duyet đơn đặt hàng từ khách hàng, quản lý chương trình khuyến mãi, xuất báo cáo doanh thu và đăng tải bài viết/tin tức. Hệ thống phân quyền thành 2 role: Admin (toàn quyền) và Editor (quyền hạn chế, chỉ quản lý nội dung).

2.2 Yêu cầu chức năng

2.2.1 Chức năng trang công khai (Landing Page)

Bảng II.1 liệt kê đầy đủ các chức năng của trang công khai mà khách hàng có thể truy cập mà không cần đăng nhập.

Bảng II.1: Bảng chức năng trang công khai (Landing Page)

STT	Chức năng	Mô tả chi tiết
1	Trang chủ	Hero banner giới thiệu, hiển thị danh mục dịch vụ nổi bật (lấy từ API), cam kết Uptime 99.99%, tin tức mới nhất, testimonials khách hàng.
2	Giới thiệu	Lịch sử công ty, hạ tầng datacenter, các chứng chỉ (ISO), cam kết SLA/Uptime 99.99%, đội ngũ kỹ thuật.
3	Dịch vụ	Danh mục: VPS, Hosting, Domain, Email doanh nghiệp, SSL, Firewall chống DDoS. Mỗi dịch vụ kèm mô tả và thông số kỹ thuật chi tiết.
4	Bảng giá	So sánh các gói theo cấu hình (CPU/RAM/SSD/Bảng thông). Chuyển đổi giá theo chu kỳ Tháng/Năm (toggle). Hiển thị mã QR cho từng gói dịch vụ.
5	Khách hàng	Đánh giá/testimonial từ khách hàng, logo đối tác tiêu biểu.
6	Tin tức / Blog	Danh sách bài viết với phân trang, tìm kiếm theo từ khóa, phân loại (hướng dẫn, khuyến mãi). Trang chi tiết bài viết theo slug.
7	Liên hệ / Đặt DV	Form đăng ký: chọn chủ đề dịch vụ, nhập thông tin khách hàng. Dữ liệu được lưu vào Database qua API.
8	Đối tác / Affiliate	Trang thông tin chính sách hoa hồng, danh sách đối tác chiến lược. Form đăng ký làm đối tác/affiliate gửi qua API.

2.2.2 Chức năng trang quản trị (Admin Dashboard)

Bảng II.2 liệt kê các chức năng quản trị, yêu cầu đăng nhập JWT và phân quyền Role.

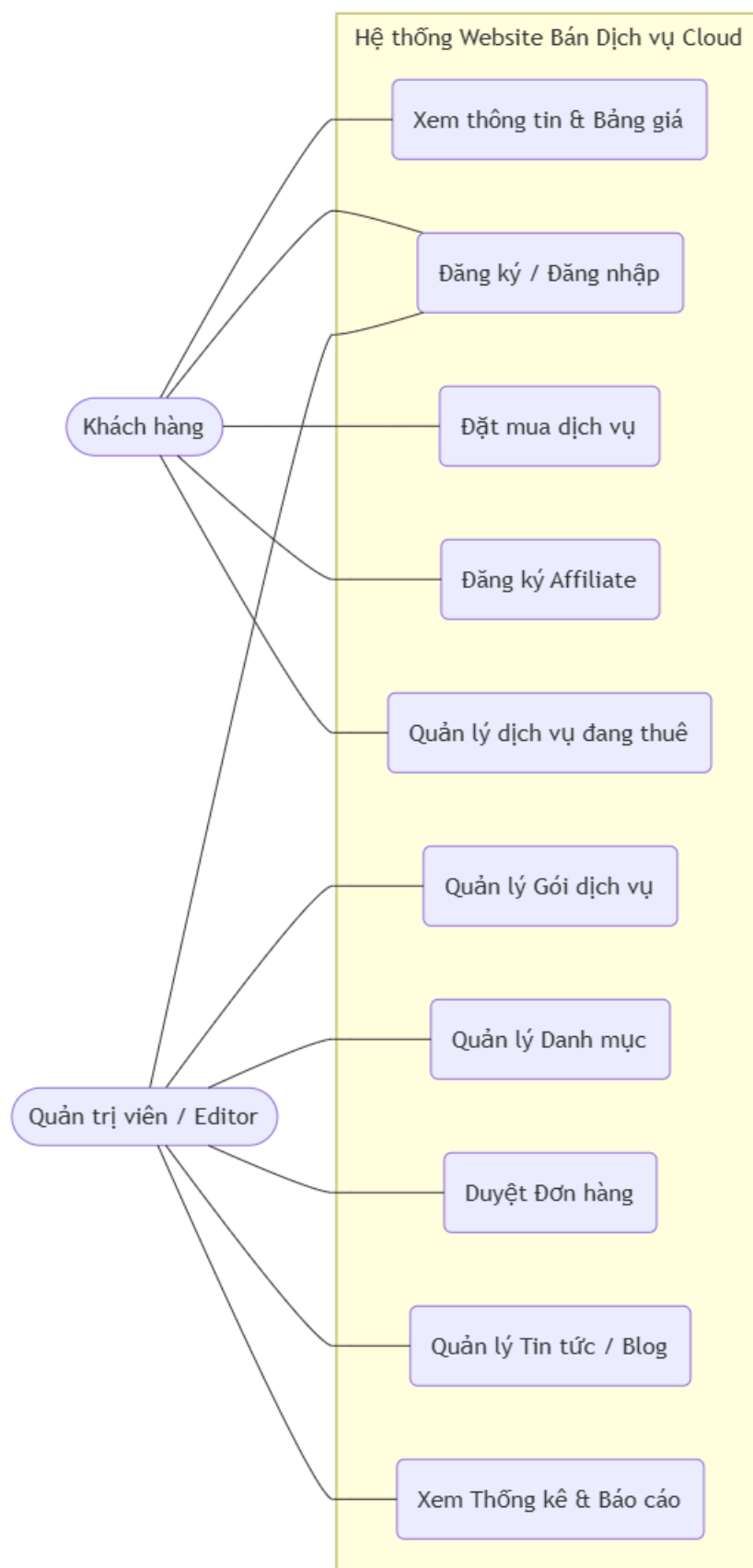
Bảng II.2: Bảng chức năng trang quản trị (Admin Dashboard)			
STT	Chức năng	Mô tả chi tiết	Role
1	Đăng nhập, Refresh Token, Đổi mật khẩu	Xác thực bằng JWT. Hỗ trợ Refresh Token gia hạn phiên và đổi mật khẩu cho tài khoản đang đăng nhập.	Tất cả
2	CRUD Gói dịch vụ	Thêm, sửa, xóa các gói dịch vụ (CPU, RAM, Disk, giá). Tự động cập nhật giá trên trang công khai.	Admin
3	CRUD Danh mục DV + Sinh QR	Quản lý danh mục (VPS, Hosting, Domain). Sinh mã QR cho từng gói, quét ra trang chi tiết.	Admin
4	CRUD Tin tức/Blog	Soạn thảo, chỉnh sửa, xóa tin tức. Hỗ trợ gắn slug URL.	Admin, Editor
5	Quản lý Đơn hàng + Affiliate	Xem danh sách, đổi trạng thái (Mới → Đang xử lý → Hoàn tất/Từ chối).	Admin, Editor
6	CRUD Khuyến mãi	Thêm, sửa, xóa chương trình khuyến mãi với mã code, phần trăm giảm giá, thời hạn.	Admin
7	Thống kê biểu đồ	Tổng quan: số đơn hàng, doanh thu theo tháng, gói dịch vụ được quan tâm. Biểu đồ trực quan.	Admin
8	Xuất Excel	Xuất danh sách yêu cầu đặt dịch vụ ra file Excel (.xlsx) qua thư viện ClosedXML.	Admin
9	Audit Log	Ghi lại lịch sử: ai đăng nhập, ai sửa giá, khi nào. Hiển thị dạng bảng có phân trang.	Admin

2.3 Yêu cầu phi chức năng

- **Giao diện (UI/UX):** Website phải có thiết kế hiện đại, tốc độ phản hồi nhanh (dưới 3 giây) và hiển thị tốt trên cả nền tảng Desktop và Mobile (Responsive). Menu điều hướng hỗ trợ Hamburger menu trên thiết bị di động.
- **Bảo mật:** Mật khẩu phải được mã hóa bằng thuật toán BCrypt trước khi lưu trữ. Các API quản trị yêu cầu xác thực JWT nghiêm ngặt với phân quyền theo Role (`[Authorize(Roles = "Admin")]`).
- **Kiến trúc:** Hệ thống được xây dựng theo mô hình Clean Architecture 4 tầng, đảm bảo tính tách biệt (Separation of Concerns) giữa các module.
- **Tính sẵn sàng và mở rộng:** Hệ thống được đóng gói bằng Docker, chỉ cần một lệnh `docker compose up` là có thể triển khai trên mọi hệ điều hành.
- **Kiểm thử:** Tối thiểu 15 unit test cases sử dụng xUnit và Moq, đảm bảo tính đúng đắn của logic nghiệp vụ tại tầng Application.

2.4 Sơ đồ Use Case (Ca sử dụng)

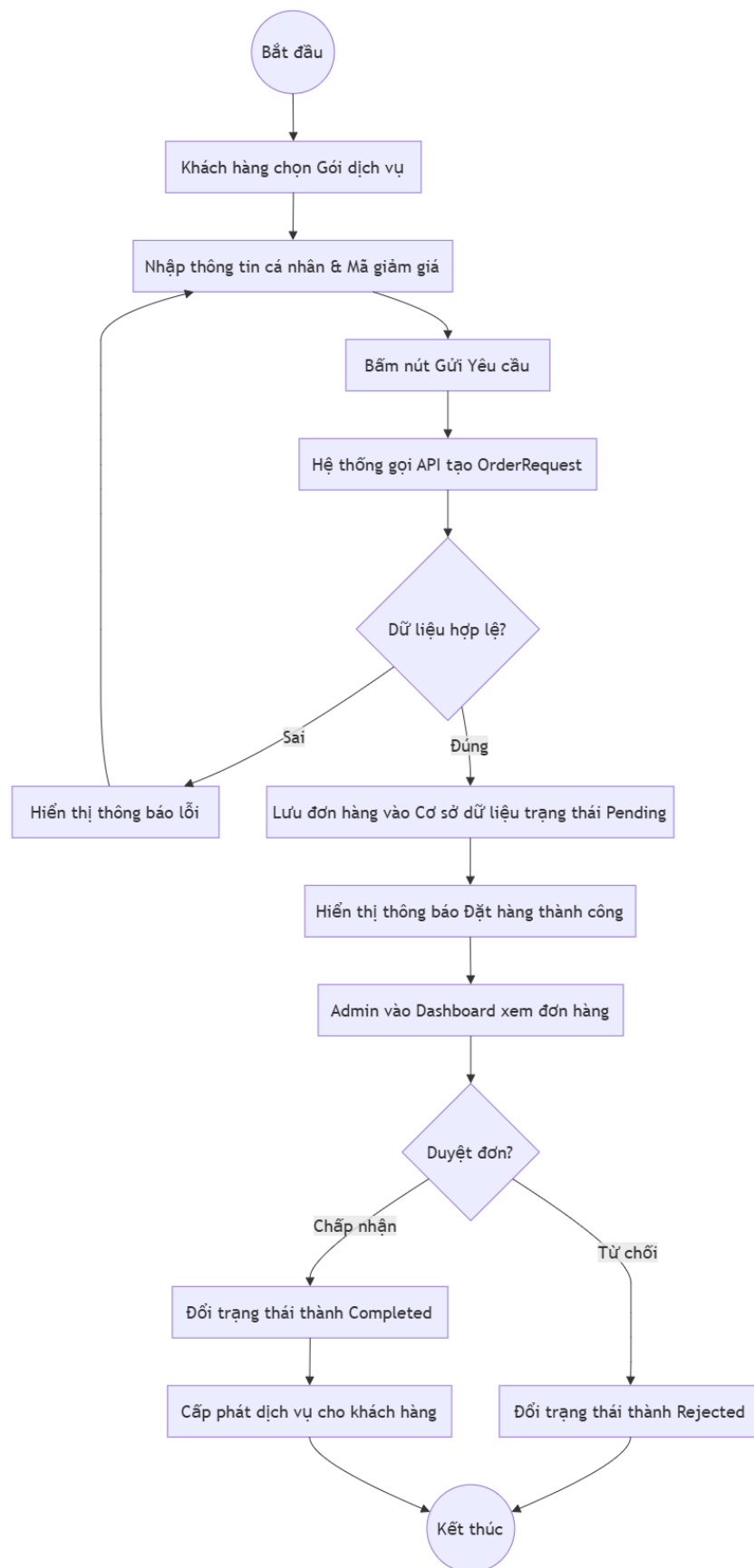
Sơ đồ Use Case tổng quát giúp hình dung rõ bức tranh toàn cảnh về cách các tác nhân tương tác với hệ thống. Khách hàng (Customer) tương tác với các trang công khai, còn Quản trị viên (Admin/Editor) tương tác với hệ thống quản trị sau khi xác thực JWT.



Hình II.1: Sơ đồ Use Case tổng quát của hệ thống Website bán dịch vụ Cloud

2.5 Sơ đồ Hoạt động (Activity Diagram)

Để làm rõ luồng nghiệp vụ cốt lõi, Sơ đồ hoạt động sau đây minh họa quy trình “Đặt hàng dịch vụ” của một khách hàng từ lúc bắt đầu cho đến khi quản trị viên phê duyệt đơn hàng.



Hình II.2: Sơ đồ Hoạt động nghiệp vụ Đặt hàng dịch vụ

CHƯƠNG III: THIẾT KẾ HỆ THỐNG

3.1 Thiết kế Kiến trúc (Architecture Design)

Để đảm bảo hệ thống có khả năng bảo trì cao, dễ dàng mở rộng và độc lập với các thư viện hoặc cơ sở dữ liệu, dự án được xây dựng dựa trên nguyên lý của mô hình **Clean Architecture** [2]. Mã nguồn Backend chia thành 4 project (4 tầng đồng tâm):

1. **CloudService.Domain (Core):** Chứa các logic nghiệp vụ cốt lõi, hoàn toàn thuần C#. Gồm 9 thực thể (Entities), 3 Enums (UserRole, OrderStatus, BillingCycle), lớp cơ sở BaseEntity và các giao diện Repository. Lớp này **không tham chiếu** đến bất kỳ lớp nào khác.
2. **CloudService.Application:** Đóng vai trò làm lớp trung gian (Use Cases), chứa 8 Services (OrderService, CategoryService, NewsArticleService, ServicePlanService, AffiliateService, AuditLogService, AuthService, AdminStatsService) cùng 9 giao diện (Interfaces) tương ứng và các DTOs (Data Transfer Objects).
3. **CloudService.Infrastructure:** Thực thi (Implement) các giao diện được định nghĩa ở lớp Core. Chứa: AppDbContext (DbContext của EF Core), 7 Repository cụ thể, lớp UnitOfWork, và 5 Infrastructure Services (JwtService, PasswordHashService, QrCodeService, ExcelExportService, AuthService).
4. **CloudService.WebApi (Presentation):** Là lớp giao tiếp với bên ngoài (Frontend). Chứa 14 Controllers (5 Public + 9 Admin), 2 Middleware (ExceptionMiddleware, AuditMiddleware) và cấu hình Dependency Injection tại Program.cs.

3.2 Thiết kế Cơ sở dữ liệu (Database Design)

Dữ liệu của hệ thống được thiết kế chặt chẽ và chuẩn hóa nhằm tránh dư thừa thông tin. Hệ thống sử dụng tổng cộng **10 bảng** (tương ứng 9 Entities + 1 bảng Role tích hợp Enum).

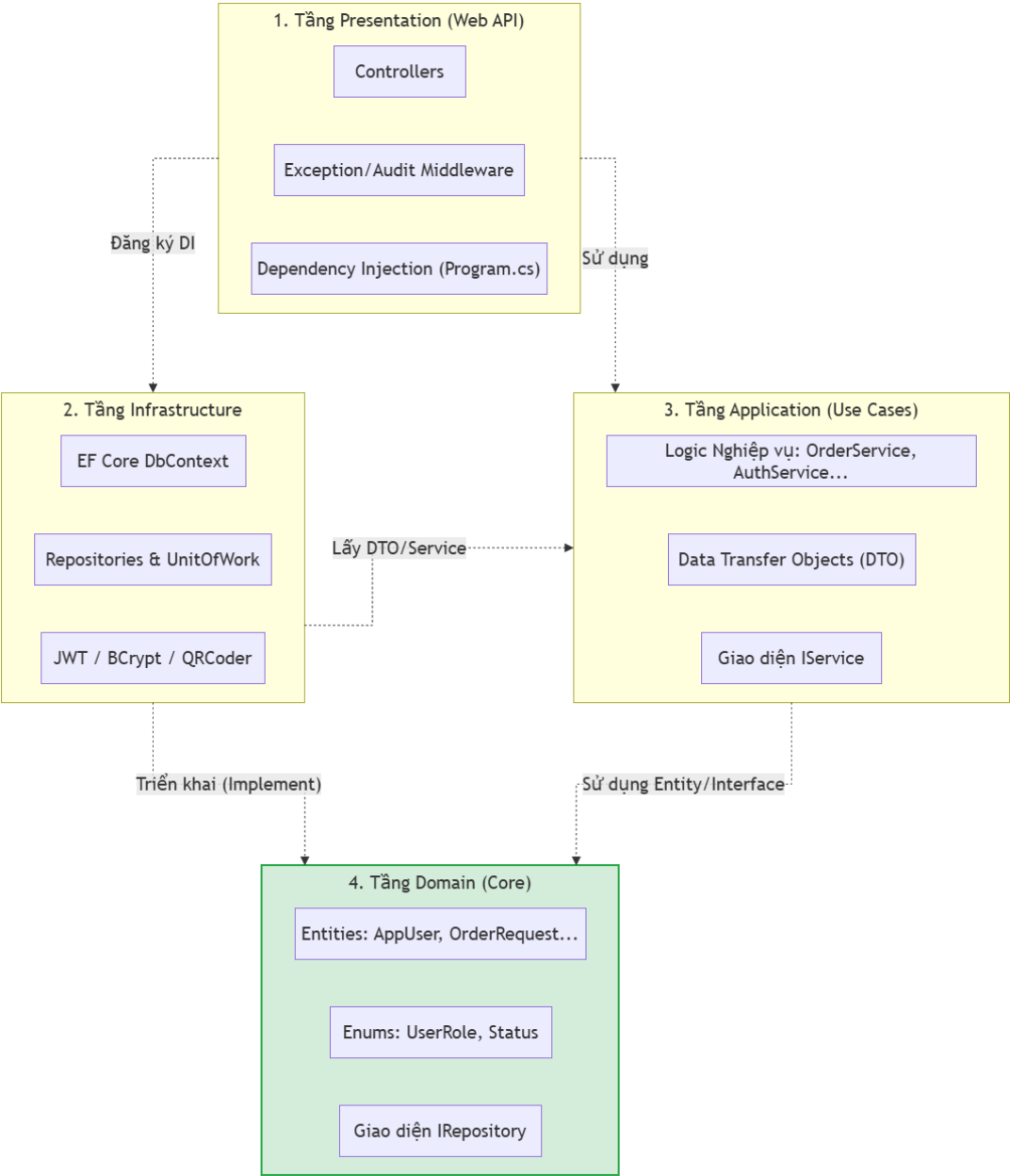
3.2.1 Sơ đồ thực thể liên kết (ERD)

3.2.2 Mô tả các thực thể (Entities) chính

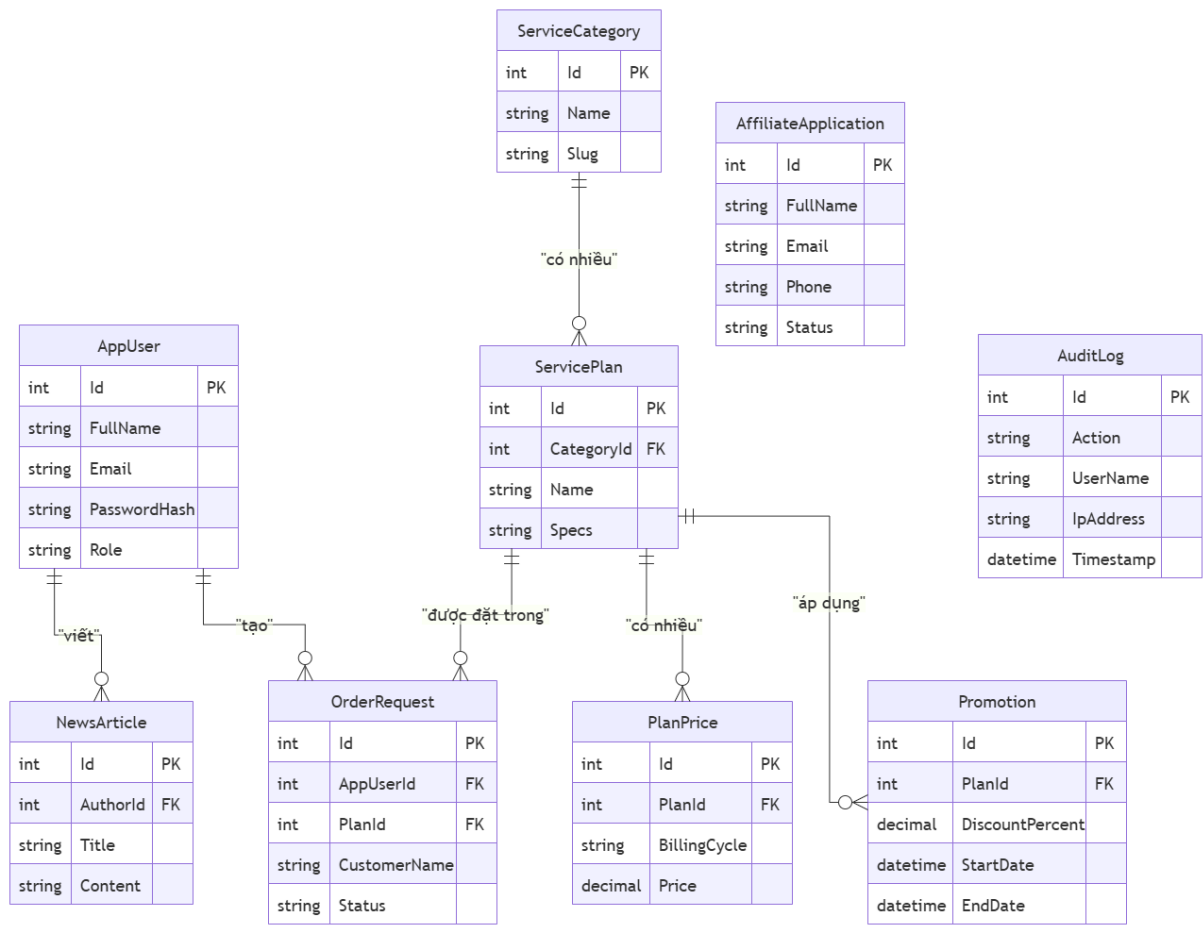
Bảng III.1 mô tả chi tiết 9 thực thể chính được triển khai trong hệ thống, tất cả đều kế thừa từ BaseEntity (chứa Id, CreatedAt, UpdatedAt).

3.3 Thiết kế RESTful API

Backend cung cấp tổng cộng **37 endpoints** chia thành 3 nhóm chính. Bảng III.2 liệt kê các endpoint tiêu biểu.



Hình III.1: Sơ đồ khối Clean Architecture trong dự án



Hình III.2: Sơ đồ thực thể liên kết (ERD)

Bảng III.1: Danh sách các thực thể (Entities) trong hệ thống

STT	Entity	Mô tả và các thuộc tính chính
1	ServiceCategory	Danh mục dịch vụ gốc (VPS, Hosting, Domain). Thuộc tính: Name, Slug, Description. Quan hệ 1-N với ServicePlan.
2	ServicePlan	Gói dịch vụ cụ thể. Thuộc tính: Name, Slug, Specs (CPU/RAM/SSD), CategoryId. Quan hệ N-1 với ServiceCategory, 1-N với PlanPrice.
3	PlanPrice	Bảng giá theo chu kỳ. Thuộc tính: BillingCycle (Monthly/Yearly), Price, PlanId. Cho phép một gói có nhiều mức giá khác nhau.
4	Promotion	Chương trình khuyến mãi. Thuộc tính: PlanId, DiscountPercent, StartDate, EndDate, IsActive. Quan hệ N-1 với ServicePlan.
5	NewsArticle	Bài viết/Tin tức. Thuộc tính: Title, Slug, Content (HTML), Summary, Category, AuthorId.
6	OrderRequest	Yêu cầu đặt dịch vụ. Thuộc tính: CustomerName, Email, Phone, ServiceName, PlanId, BillingCycle, Status (Enum: Pending/Processing/Completed/Rejected).
7	AffiliateApplication	Đơn đăng ký đối tác. Thuộc tính: FullName, Email, Phone, Website, Status.
8	AppUser	Tài khoản quản trị. Thuộc tính: FullName, Email, PasswordHash, Role (Enum: Admin/Editor), RefreshToken, RefreshTokenExpiry.
9	AuditLog	Nhật ký hệ thống. Thuộc tính: Action, Username, IpAddress, Timestamp, Details.

Bảng III.2: Danh sách các API Endpoint chính

Method	Route	Mô tả
<i>Public API (Không cần đăng nhập)</i>		
GET	/api/public/service-categories	Danh mục dịch vụ
GET	/api/public/service-plans	Tất cả gói dịch vụ
GET	/api/public/service-plans/{id}/qr	Mã QR cho gói DV
GET	/api/public/news	Danh sách tin tức
GET	/api/public/news/{slug}	Chi tiết bài viết
POST	/api/public/orders	Gửi yêu cầu đặt DV
POST	/api/public/affiliates	Đăng ký affiliate
<i>Auth API</i>		
POST	/api/auth/login	Đăng nhập → JWT
POST	/api/auth/refresh	Gia hạn Access Token
POST	/api/auth/change-password	Đổi mật khẩu
<i>Admin API (Yêu cầu JWT + Role)</i>		
CRUD	/api/admin/service-plans	Quản lý gói DV
CRUD	/api/admin/categories	Quản lý danh mục
CRUD	/api/admin/news	Quản lý tin tức
CRUD	/api/admin/promotions	Quản lý khuyến mãi
GET	/api/admin/stats/summary	Thống kê tổng quan
GET	/api/admin/export/orders	Xuất Excel đơn hàng
GET	/api/admin/audit-logs	Nhật ký hệ thống

3.4 Thiết kế Mẫu thiết kế (Design Patterns)

Các mẫu thiết kế (Design Patterns) [1] là những giải pháp tối ưu đã được kiểm chứng cho các vấn đề lặp lại trong thiết kế phần mềm. Dự án áp dụng các pattern sau:

3.4.1 Repository Pattern

Lớp `IGenericRepository<T>` cung cấp một tập hợp các hàm CRUD tiêu chuẩn chung cho mọi bảng. Các lớp nghiệp vụ chỉ cần gọi các phương thức như `GetAllAsync`, `GetByIdAsync` mà không cần biết chi tiết về câu lệnh SQL dưới tầng hạ tầng. Điều này giúp mã nguồn Application hoàn toàn không bị “trói buộc” với Entity Framework.

Trong dự án, 7 Repository cụ thể đã được triển khai: `ServiceCategoryRepository`, `ServicePlanRepository`, `NewsArticleRepository`, `OrderRequestRepository`, `AffiliateApplicationRepository`, `AppUserRepository`, `AuditLogRepository`.

3.4.2 Unit of Work Pattern

Khi có nhiều Repositories cần tương tác với Database cùng một lúc (ví dụ: tạo đơn hàng `OrderRequest` và cập nhật trạng thái affiliate), mô hình Unit of Work (giao diện `IUnitOfWork`) được sử dụng để gom các thao tác này vào chung một Giao dịch (Transaction). Lệnh `CompleteAsync()` đảm bảo dữ liệu chỉ được lưu nếu không xảy ra lỗi.

3.4.3 Middleware Pattern

Hệ thống sử dụng 2 Middleware tùy chỉnh theo dạng “ống nước” (Pipeline):

- `ExceptionMiddleware`: Bắt toàn bộ exception chưa được xử lý và trả về response JSON chuẩn với mã lỗi HTTP phù hợp (400, 404, 500).

- **AuditMiddleware:** Tự động ghi lại mọi hành động quan trọng (đăng nhập, thay đổi dữ liệu) vào bảng **AuditLog** kèm thông tin IP, thời gian và người thực hiện.

3.4.4 Dependency Injection (DI)

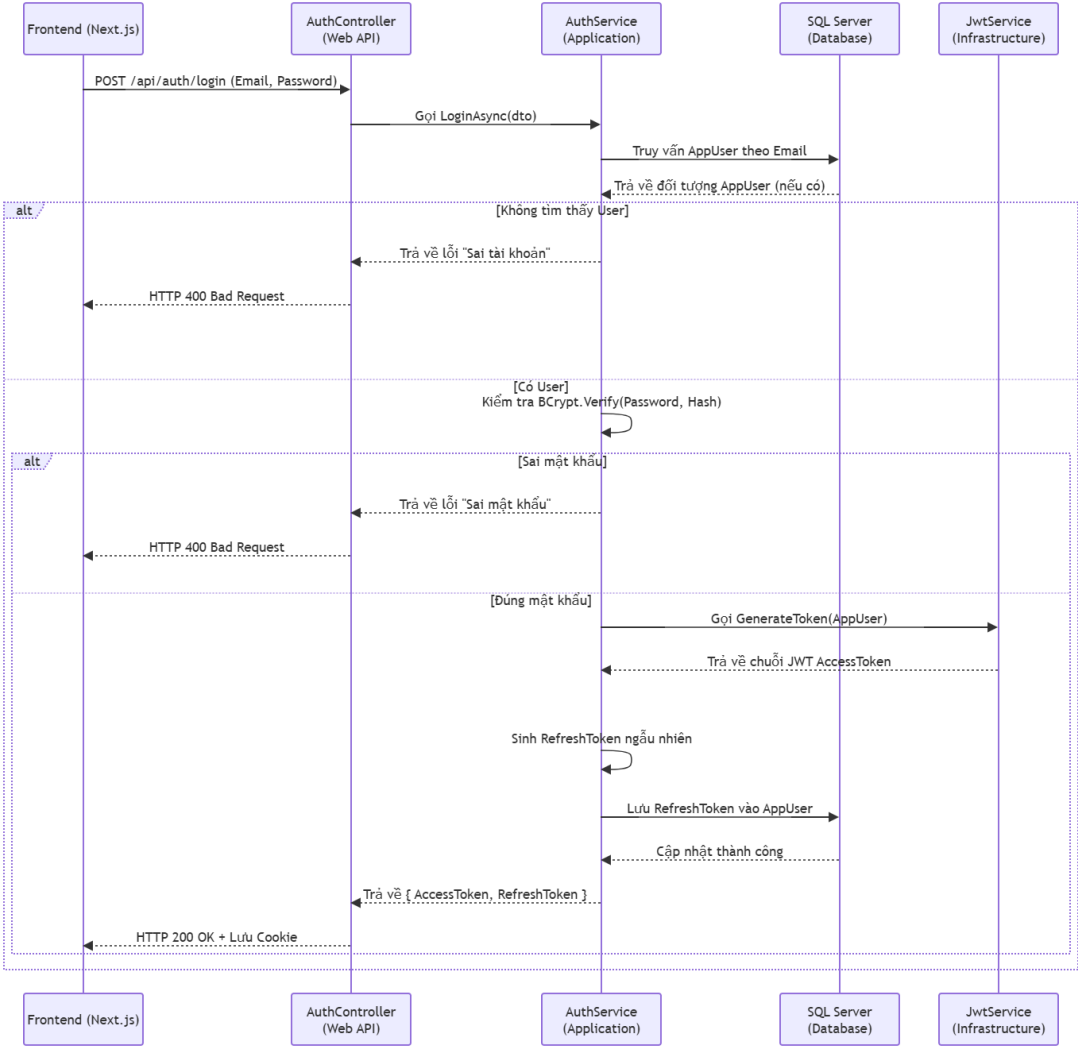
Toàn bộ việc “nối dây” giữa các Interface và Implementation được cấu hình tập trung tại `Program.cs` thông qua DI container tích hợp sẵn của ASP.NET Core. Ví dụ:

```
builder.Services.AddScoped<IOrderService, OrderService>();  
builder.Services.AddScoped<ICategoryService, CategoryService>();  
builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
```

3.5 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ tuần tự thể hiện sự giao tiếp và thứ tự các thông điệp truyền đi giữa Client (Frontend) và Server (Backend) trong một chức năng.

Dưới đây minh họa quy trình Đăng nhập và nhận JWT Token: Frontend gửi `POST /api/auth/login` → `AuthController` gọi `AuthService` → kiểm tra `PasswordHash` bằng `BCrypt` → `JwtService` sinh `AccessToken` + `RefreshToken` → trả về Frontend lưu Cookie.



Hình III.3: Sơ đồ Tuần tự luồng xử lý xác thực đăng nhập cấp phát JWT

CHƯƠNG IV: XÂY DỰNG VÀ KIỂM THỬ HỆ THỐNG

4.1 Xây dựng Giao diện người dùng (UI)

Giao diện của hệ thống được xây dựng hoàn toàn bằng Next.js (App Router) và Tailwind CSS, mang lại trải nghiệm mượt mà, phản hồi siêu tốc. Toàn bộ giao diện sử dụng thiết kế Responsive, tương thích trên cả Desktop và Mobile.

4.1.1 Cấu trúc thư mục Frontend

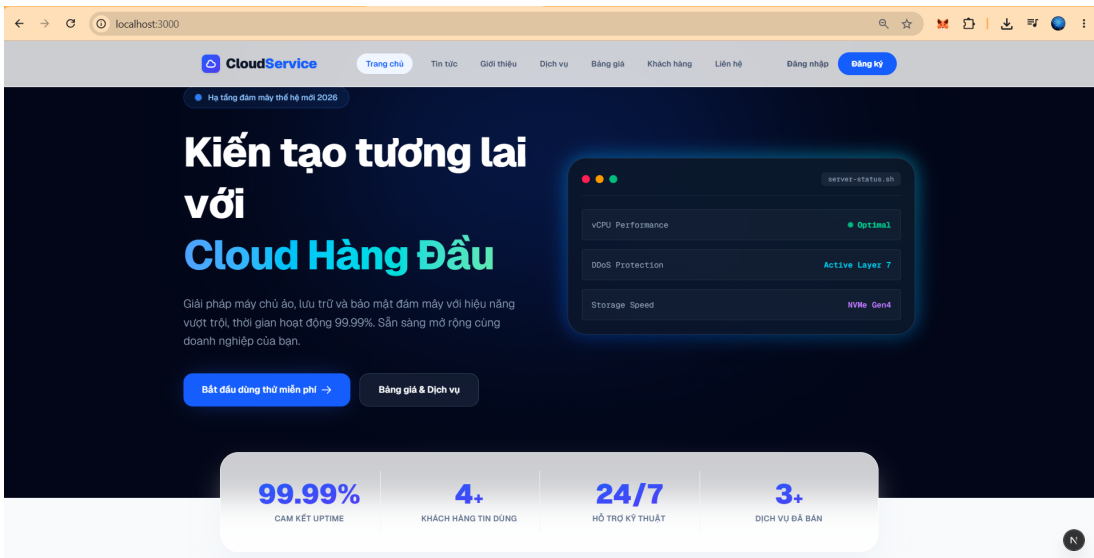
Mã nguồn Frontend được tổ chức theo kiến trúc App Router của Next.js:

```
frontend/src/
|-- app/
|   |-- (public)/           # 10 trang công khai
|   |   |-- page.tsx        # Trang chủ (SSR)
|   |   |-- bang-gia/       # Bảng giá + QR
|   |   |-- tin-tuc/        # Tin tức + [slug]
|   |   |-- dich-vu/        # Dịch vụ + [slug]
|   |   |-- lien-he/        # Form liên hệ -> API
|   |   |-- doi-tac/        # Form affiliate -> API
|   |   |-- Header.tsx     # Navbar + Hamburger
|   |   |-- Footer.tsx     # Footer
|   |-- admin/             # 8 trang quản trị
|   |   |-- dashboard/     # Thông kê tổng quan
|   |   |-- services/      # CRUD gọi dịch vụ
|   |   |-- news/          # CRUD tin tức
|   |   |-- orders/        # Quản lý đơn hàng
|   |   |-- promotions/    # CRUD khuyến mãi
|   |   |-- audit-logs/    # Nhật ký hệ thống
|   |-- layout.tsx         # Root layout + SEO
|-- lib/axios.ts           # apiClient cấu hình
|-- middleware.ts          # Auth middleware
-- types/index.ts          # TypeScript interfaces
```

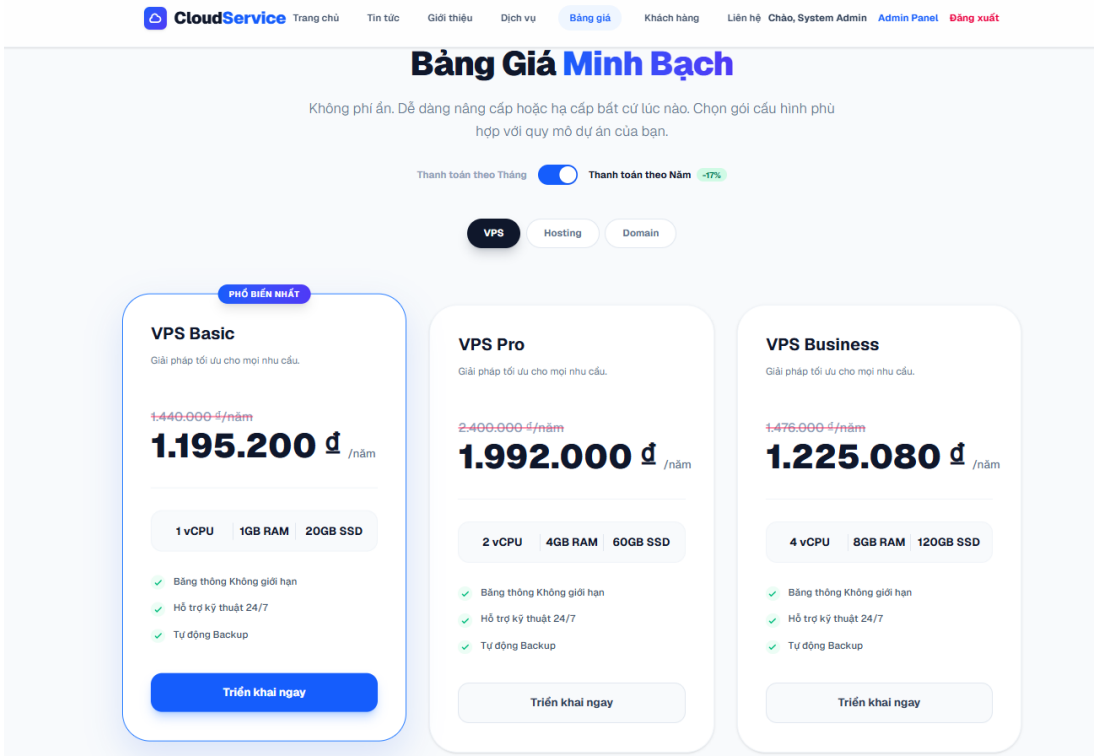
4.1.2 Giao diện Landing Page công khai

Trang Landing Page tập trung vào việc hiển thị các thông tin giới thiệu, các lợi thế cạnh tranh (như cam kết Uptime 99.99%) và bảng giá dịch vụ để thu hút người mua.

Các trang công khai chính gồm: Trang chủ, Giới thiệu, Dịch vụ (danh sách + chi tiết theo slug), Bảng giá (toggle Tháng/Năm + mã QR), Khách hàng (testimonials), Tin tức (danh sách + chi tiết), Liên hệ (form gửi API) và Đối tác (form đăng ký affiliate).



Hình IV.1: Giao diện trang chủ (Landing Page)



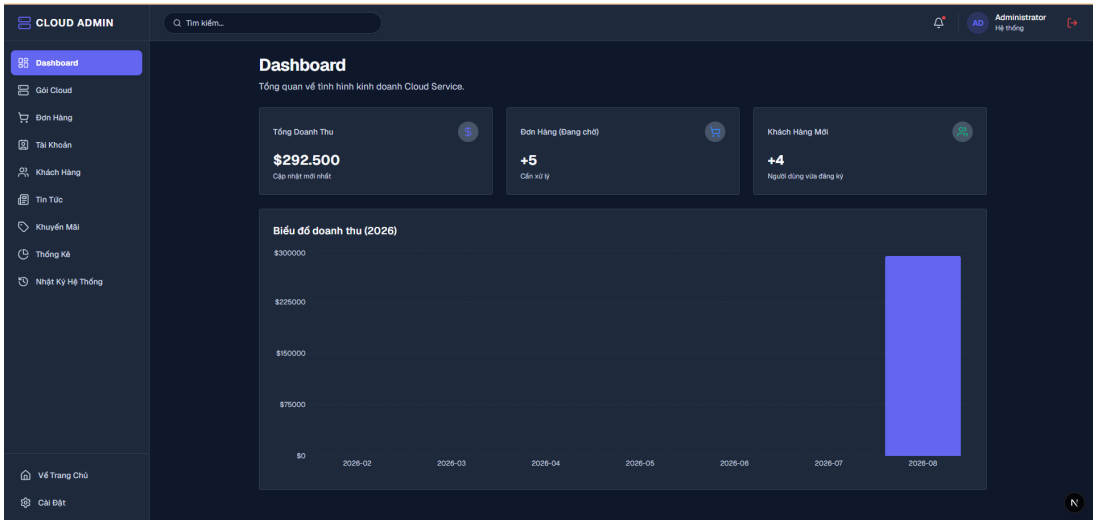
Hình IV.2: Giao diện trang Bảng giá – Chuyển đổi Tháng/Năm và hiển thị mã QR

4.1.3 Giao diện Quản trị (Admin Dashboard)

Sau khi xác thực thành công qua JWT tại trang /login, hệ thống dựa vào Role của người dùng để điều hướng đến Admin Dashboard. Giao diện quản trị bao gồm sidebar navigation cố định bên trái và khu vực nội dung chính bên phải.

Các trang Admin chính bao gồm:

- **Dashboard:** Hiển thị thông kê tổng quan (tổng đơn hàng, doanh thu, số bài viết) qua các thẻ card và biểu đồ doanh thu theo tháng, dữ liệu lấy từ API /admin/stats/summary và /admin/stats/revenue-chart.
- **Quản lý Tin tức:** Hỗ trợ CRUD đầy đủ (Thêm/Sửa/Xóa bài viết) qua modal form, dữ liệu đồng bộ real-time với Backend.
- **Quản lý Đơn hàng:** Hiển thị danh sách đơn hàng, cho phép thay đổi trạng thái (Pending → Processing → Completed/Rejected) và xuất ra file Excel.
- **Quản lý Khuyến mãi:** Tạo và xóa chương trình khuyến mãi (mã code, phần trăm giảm, thời hạn).
- **Audit Logs:** Xem lịch sử hành động của hệ thống (ai đăng nhập, từ IP nào, lúc mấy giờ).



Hình IV.3: Giao diện Bảng điều khiển Quản trị viên (Admin Dashboard)

4.2 Kiểm thử đơn vị (Unit Testing)

Kiểm thử phần mềm là một khâu không thể thiếu để đảm bảo độ tin cậy của sản phẩm. Dự án đã viết mã kiểm thử tự động sử dụng framework **xUnit** và thư viện mock **Moq** trên .NET trong project riêng biệt `CloudService.UnitTests`.

4.2.1 Tổng quan kết quả

Tổng cộng **16 test cases** (vượt yêu cầu tối thiểu 15) đã được viết và chạy thành công, phân bổ theo 5 file test:

4.2.2 Phương pháp Mock

Mỗi test case sử dụng thư viện **Moq** để tạo đối tượng giả lập (mock) cho các Repository và UnitOfWork, giúp cô lập logic nghiệp vụ tại tầng Application mà không cần kết nối đến cơ sở dữ liệu thực. Ví dụ trích code test:

Bảng IV.1: Chi tiết kết quả Unit Test

STT	File Test	Số test	Đối tượng kiểm thử
1	OrderServiceTests.cs	6	Tạo đơn hàng, đổi trạng thái, validate input, lấy danh sách, xử lý lỗi
2	ServicePlanServiceTests.cs	5	CRUD gói dịch vụ, tính giá theo chu kỳ, validate specs
3	AffiliateServiceTests.cs	2	Tạo đơn đăng ký affiliate, đổi trạng thái
4	CategoryServiceTests.cs	2	CRUD danh mục, kiểm tra trùng tên
5	NewsArticleServiceTests.cs	1	Tạo bài viết mới, generate slug
Tổng cộng		16	Tất cả PASSED

[Fact]

```

public async Task CreateOrder_WithValidData_ReturnsSuccess()
{
    // Arrange - Tạo mock repository
    var mockRepo = new Mock<IOrderRequestRepository>();
    var mockUoW = new Mock<IUnitOfWork>();
    mockUoW.Setup(u => u.Orders).Returns(mockRepo.Object);

    var service = new OrderService(mockUoW.Object);
    var dto = new CreateOrderDto {
        CustomerName = "Nguyen Van A",
        Email = "test@email.com",
        ServiceName = "VPS Pro"
    };

    // Act - Gọi hàm cần test
    var result = await service.CreateAsync(dto);

    // Assert - Kiểm tra kết quả
    Assert.NotNull(result);
    mockUoW.Verify(u => u.CompleteAsync(), Times.Once);
}

```

```

Problems 56 Output Debug Console Terminal Ports
PS D:\PTPM_huong_doi_tuong\BTL_Ban_dich_vu_cloud> dotnet test backend/CloudService.UnitTests/CloudService.UnitTests.csproj
>>
Restore complete (0.4s)
  CloudService.Domain succeeded (0.4s) -> backend\CloudService.Domain\bin\Debug\net9.0\CloudService.Domain.dll
  CloudService.Application succeeded (0.4s) -> backend\CloudService.Application\bin\Debug\net9.0\CloudService.Application.dll
  CloudService.UnitTests succeeded (0.5s) -> backend\CloudService.UnitTests\bin\Debug\net9.0\CloudService.UnitTests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.9)
[xUnit.net 00:00:00.52] Discovering: CloudService.UnitTests
[xUnit.net 00:00:00.55] Discovered: CloudService.UnitTests
[xUnit.net 00:00:00.55] Starting: CloudService.UnitTests
[xUnit.net 00:00:01.04] Finished: CloudService.UnitTests
CloudService.UnitTests test succeeded (3.0s)

Test summary: total: 16, failed: 0, succeeded: 16, skipped: 0, duration: 3.0s
Build succeeded in 5.3s
PS D:\PTPM_huong_doi_tuong\BTL_Ban_dich_vu_cloud>

```

Hình IV.4: Kết quả chạy 16 Unit Test toàn bộ PASSED

4.3 Triển khai liên tục (CI/CD) và Docker

Dự án áp dụng quy trình Phát triển phần mềm hiện đại, tuân thủ nguyên tắc “Cộng tác chặt chẽ”. Mã nguồn được quản lý phiên bản qua Git trên GitHub.

4.3.1 Luồng làm việc với GitHub Actions (CI)

Mọi thao tác đẩy mã (Push) lên nhánh `develop` hoặc `main`, và mọi Pull Request đều kích hoạt pipeline GitHub Actions. Nội dung pipeline `.github/workflows/ci.yml`:

1. **Checkout:** Kéo mã nguồn từ repository.
2. **Setup .NET 9:** Cài đặt môi trường .NET SDK phiên bản 9.0.x.
3. **Restore:** Khôi phục thư viện NuGet cho cả API và Unit Test project.
4. **Build:** Biên dịch mã nguồn ở chế độ Release.
5. **Test:** Chạy toàn bộ 16 Unit Tests. Nếu bất kỳ test nào thất bại, pipeline sẽ báo Failed và không cho phép merge PR.

4.3.2 Đóng gói ứng dụng với Docker

Để giải quyết bài toán “Chạy được trên máy tôi nhưng lỗi trên máy chủ”, toàn bộ hệ thống được cấu hình vào các Container riêng biệt:

- **Dockerfile (Backend):** Multi-stage build: stage 1 (build) sử dụng image `dotnet/sdk:9.0` để biên dịch, stage 2 (runtime) sử dụng image `dotnet/aspnet:9.0` gọn nhẹ để chạy.
- **docker-compose.yml:** Định nghĩa 2 services:
 - `sqlldb`: SQL Server 2022 Express với dữ liệu lưu trên volume `sql_data`.
 - `backend`: API .NET 9 kết nối tới container `sqlldb`, expose port 5000.

Chỉ với một dòng lệnh duy nhất `docker compose up -d`, hệ thống đã được triển khai sẵn sàng trên mọi hệ điều hành hoặc máy chủ Cloud thực tế.

4.3.3 Quy trình làm việc nhóm với Git

Nhóm áp dụng mô hình **Feature Branch Workflow**:

1. Mỗi tính năng được phát triển trên một nhánh riêng (ví dụ: `feature/admin-dashboard`).
2. Khi hoàn thành, tạo Pull Request (PR) lên nhánh `develop`.
3. Các thành viên khác review code và approve.
4. CI pipeline tự động chạy build + test. Nếu PASSED, PR được phép merge.
5. Nhánh `main` chỉ chứa code ổn định, sẵn sàng triển khai.

CHƯƠNG V: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Đánh giá kết quả đạt được

Kết thúc quá trình thực hiện đồ án môn học Phát triển phần mềm hướng đối tượng, nhóm đã hoàn thành các mục tiêu và yêu cầu từ giảng viên đề ra. Những kết quả nổi bật bao gồm:

5.1.1 Khía cạnh Kỹ thuật

- Làm chủ và ứng dụng nhuần nhuyễn mô hình **Clean Architecture** đa tầng (4 project: Domain, Application, Infrastructure, WebApi).
- Khai thác hiệu quả các nguyên lý thiết kế **SOLID** và **Design Patterns** (Repository, Unit of Work, Middleware, Dependency Injection) để tạo ra một lõi phần mềm Backend linh hoạt, dễ nâng cấp.
- Xây dựng **37 RESTful API endpoints** hoàn chỉnh phục vụ cả trang công khai và trang quản trị.
- Phát triển **10 trang Landing Page + 8 trang Admin Dashboard** với giao diện hiện đại sử dụng Next.js App Router và Tailwind CSS.

5.1.2 Khía cạnh Nghiệp vụ

- Xây dựng thành công hệ thống website bán dịch vụ Cloud (VPS, Hosting, Domain) bao gồm toàn bộ luồng nghiệp vụ từ người dùng tra cứu, đăng ký đến phía admin phê duyệt và thống kê.
- Hệ thống xử lý tốt cơ chế bảo mật: xác thực JWT với AccessToken + RefreshToken, mã hóa mật khẩu bằng BCrypt, phân quyền theo Role (Admin, Editor).
- Tích hợp thành công các tính năng nâng cao: sinh mã QR cho gói dịch vụ, xuất báo cáo Excel, ghi nhật ký hệ thống (Audit Log).

5.1.3 Khía cạnh Quy trình

- Vận dụng thực tế mô hình phát triển phần mềm thông qua việc chia nhỏ tính năng trên feature branch.
- Tích hợp luồng CI/CD (GitHub Actions) tự động build và chạy **16 Unit Tests** (vượt yêu cầu tối thiểu 15).
- Chuẩn hóa triển khai môi trường qua Docker: Dockerfile cho Backend và docker-compose.yml (API + SQL Server 2022).

Dự án không chỉ là minh chứng cho lượng kiến thức được lĩnh hội trong môn học mà còn giúp nhóm rèn luyện khả năng phối hợp làm việc thực tế, xử lý các luồng hợp nhất mã nguồn (Merge/Pull Request) trên Git.

5.2 Bảng tổng hợp kết quả đối chiếu đề bài

Bảng V.1 đối chiếu các tiêu chí chấm điểm từ đề bài với kết quả đạt được thực tế.

Bảng V.1: Đối chiếu kết quả đạt được với yêu cầu đề bài		
Tiêu chí	Điểm	Kết quả đạt được
Clean Architecture + SOLID + Design Patterns	20	4 tầng, 5 SOLID, 4 Patterns
Backend API đầy đủ, REST, ORM	20	37 endpoints, EF Core, SQL Server
Frontend đầy đủ, responsive	15	18+ trang, Responsive, SEO
Bảo mật: JWT + Role, Hash, QR	10	JWT + Refresh, BCrypt, QR Code
Unit Testing $\geq$ 15 tests	10	16 tests (xUnit + Moq)
Git + CI/CD + Docker	10	GitHub Actions, Docker Compose
Báo cáo + Thuyết trình	10	Báo cáo LaTeX 5 chương
Vấn đáp	5	(Tại buổi demo)

5.3 Phân công công việc

Để có được một dự án phần mềm hoàn thiện, nhóm đã có sự phân bổ hợp lý nhiệm vụ, phát huy thế mạnh của từng thành viên:

STT	Thành viên	Công việc phụ trách
1	[Nguyễn Trung Hậu]	Team Lead / DevOps: Phân tích yêu cầu, thiết kế CSDL (ERD). Xây dựng Backend tầng Domain và Application. Cấu hình Docker, CI/CD (GitHub Actions). Viết Unit Test (16 test cases).
2	[Lương Quang Khải]	Backend Developer: Xây dựng RESTful API (14 Controllers, 37 endpoints). Triển khai xác thực JWT (Login, Refresh Token, Change Password), BCrypt hash, QR Code, Excel Export. Cấu hình Middleware (Audit, Exception).
3	[Võ Phụng Tiên]	Frontend Developer (Public): Lập trình 10 trang Landing Page (Trang chủ, Giới thiệu, Dịch vụ, Bảng giá, Tin tức, Liên hệ, Đối tác, Khách hàng). Kết nối API cho form gửi đơn hàng và đăng ký affiliate. Responsive + Hamburger menu.
4	[Phan Thành Vinh]	Frontend Developer (Admin): Lập trình 8 trang Admin Dashboard (Dashboard, Services, News, Orders, Affiliates, Promotions, Analytics, Audit Logs). Kết nối API CRUD. Viết báo cáo tổng kết đồ án.

Bảng V.2: Bảng phân công công việc thực hiện dự án của nhóm

5.4 Hạn chế và Hướng phát triển trong tương lai

Vì giới hạn thời gian thực hiện, hệ thống vẫn còn một số khía cạnh có thể tiếp tục nghiên cứu và mở rộng:

- Thanh toán điện tử (E-Payment):** Hiện tại tính năng đặt hàng dừng lại ở khâu “Ghi nhận yêu cầu”, trong tương lai dự án có thể tích hợp API của các cổng thanh toán (như VNPay, MoMo) để tự động hóa khâu phê duyệt hóa đơn.
- Tối ưu hóa hiệu năng (Caching):** Áp dụng bộ đệm (như Redis) ở tầng Backend để lưu trữ nhanh các truy vấn lặp lại (Bảng giá, Tin tức), giúp giảm tải cho cơ sở dữ liệu.

- **Quản lý Vi dịch vụ (Microservices):** Nếu quy mô ứng dụng mở rộng (số lượng khách hàng lớn), kiến trúc có thể được tách nhỏ thành các services độc lập (ví dụ: Identity Service, Order Service) để linh hoạt trong việc Scale-up.
- **Logging tập trung:** Tích hợp Serilog với Seq hoặc Elasticsearch để giám sát log tập trung, giúp debug và theo dõi hiệu suất hệ thống production.
- **Đăng ký tài khoản khách hàng:** Bổ sung luồng Register/Login riêng cho khách hàng (không phải Admin) để có thể theo dõi lịch sử đơn hàng cá nhân.

Đồ án đã tạo ra một nền móng công nghệ vững chắc. Với những hướng phát triển trên, sản phẩm hoàn toàn có tiềm năng để thương mại hóa trong môi trường công nghiệp điện toán đám mây.

TÀI LIỆU THAM KHẢO

- [1] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [2] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [3] Microsoft. Asp.net core documentation, 2026.
- [4] Vercel. Next.js documentation, 2026.

PHỤ LỤC

Phụ lục A: Cấu trúc thư mục Backend

```
backend/
|-- CloudService.Domain/           # Tang Domain (Core)
|   |-- Common/
|   |   \-- BaseEntity.cs         # Lop co so (Id, CreatedAt, UpdatedAt)
|   |-- Entities/
|   |   |-- ServiceCategory.cs    # Danh muc dich vu
|   |   |-- ServicePlan.cs        # Goi dich vu
|   |   |-- PlanPrice.cs          # Gia theo chu ky
|   |   |-- Promotion.cs          # Khuyen mai
|   |   |-- NewsArticle.cs        # Bai viet/Tin tuc
|   |   |-- OrderRequest.cs       # Yeu cau dat DV
|   |   |-- AffiliateApplication.cs # Dang ky doi tac
|   |   |-- AppUser.cs            # Tai khoan quan tri
|   |   \-- AuditLog.cs           # Nhat ky he thong
|   \-- Enums/
|       |-- UserRole.cs           # Admin, Editor
|       |-- OrderStatus.cs        # Pending, Processing...
|       \-- BillingCycle.cs        # Monthly, Yearly
|
|-- CloudService.Application/      # Tang Application
|   |-- Interfaces/               # 9 giao dien (Interface)
|   |-- Services/                 # 8 Service implementations
|   \-- DTOs/                    # Data Transfer Objects
|
|-- CloudService.Infrastructure/   # Tang Infrastructure
|   |-- Data/
|   |   |-- AppDbContext.cs       # EF Core DbContext
|   |   \-- Migrations/           # Database migrations
|   |-- Repositories/             # 7 Repository + UnitOfWork
|   \-- Services/
|       |-- JwtService.cs         # Sinh JWT Token
|       |-- PasswordHashService.cs # BCrypt hash
|       |-- QrCodeService.cs      # Sinh ma QR
|       |-- ExcelExportService.cs # Xuat Excel
|       \-- AuthService.cs        # Xac thuc
|
|-- CloudService.WebApi/           # Tang Presentation
|   |-- Controllers/
```

```
| | |-- Public/                # 5 API công khai
| | \-- Admin/                # 9 API quản trị
| |-- Middleware/
| | |-- ExceptionMiddleware.cs
| | \-- AuditMiddleware.cs
| \-- Program.cs              # DI + Pipeline config
|
|-- CloudService.UnitTests/    # Unit Test project
| \-- Tests/                  # 5 file, 16 test cases
|
|-- Dockerfile                # Multi-stage build
\-- docker-compose.yml        # API + SQL Server
```

Phụ lục B: Cấu hình docker-compose.yml

```
version: '3.8'
services:
  sqldb:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: cloudservice_db
    environment:
      - ACCEPT_EULA=Y
      - SA_PASSWORD=CloudService@2026!
      - MSSQL_PID=Express
    ports:
      - "1433:1433"
    volumes:
      - sql_data:/var/opt/mssql

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: cloudservice_api
    depends_on:
      - sqldb
    ports:
      - "5000:80"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ConnectionStrings__Default=Server=sqldb;
        Database=CloudServiceDb;User=sa;
        Password=CloudService@2026!;
        TrustServerCertificate=True

volumes:
  sql_data:
```

Phụ lục C: Cấu hình GitHub Actions CI

```
name: .NET Backend CI
on:
  push:
    branches: [ "develop", "main" ]
  pull_request:
    branches: [ "develop", "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '9.0.x'
      - name: Restore API
        run: dotnet restore backend/CloudService.WebApi/
          CloudService.WebApi.csproj
      - name: Restore Unit Test
        run: dotnet restore backend/CloudService.UnitTests/
          CloudService.UnitTests.csproj
      - name: Build
        run: dotnet build backend/CloudService.WebApi/
          CloudService.WebApi.csproj --no-restore -c Release
      - name: Run 16 Unit Tests
        run: dotnet test backend/CloudService.UnitTests/
          CloudService.UnitTests.csproj --no-restore
```

Phụ lục D: Tài khoản Demo

Vai trò	Email	Mật khẩu
Admin	admin@cloudservice.vn	Admin@123
Customer	ustomer@gmail.com	Customer@123

Bảng V.3: Tài khoản demo cho hệ thống

Lưu ý: Mật khẩu đã được mã hóa BCrypt trước khi lưu vào Database. Đây chỉ là tài khoản demo phục vụ mục đích kiểm thử.

Phụ lục E: Mã nguồn dự án

Toàn bộ mã nguồn của dự án (bao gồm Backend, Frontend, file Docker và LaTeX báo cáo) được lưu trữ và quản lý phiên bản công khai trên GitHub tại địa chỉ:

- **Link Repository:** <https://github.com/TrungHauNguyen4/CloudService-BTL-Nhom7.git>